

TTDS Topic x Knowledge Point Deep Guide

Past-paper style matrix with explanations, formulas, and answer templates

Generated for TTDS revision

2026-05-21

Contents

1	How to Use This Guide	4
2	1. IR Basics and Definitions	5
2.1	1.1 Information Retrieval	5
2.2	1.2 Query, Document, Information Need	5
2.3	1.3 Effectiveness and Efficiency	5
2.4	1.4 Relevance	5
2.5	1.5 Synonymy and Polysemy	6
2.6	1.6 Bag-of-Words	6
2.7	1.7 IR Pipeline and RAG	6
3	2. Text Laws	7
3.1	2.1 Zipf' s Law	7
3.2	2.2 Zipf and Stop Words	7
3.3	2.3 Long Tail and Rare Terms	7
3.4	2.4 Heaps' Law	7
3.5	2.5 Document Order in Vocabulary Growth	8
3.6	2.6 Clumping / Contagion	8
4	3. Preprocessing	9
4.1	3.1 Preprocessing Pipeline	9
4.2	3.2 Tokenisation	9
4.3	3.3 Stopword Removal	9
4.4	3.4 Removing URLs or Non-Alphanumeric Characters	9
4.5	3.5 Case Folding and Normalisation	9
4.6	3.6 Stemming	10
4.7	3.7 Language-Specific Preprocessing	10
4.8	3.8 Preprocessing Cannot Solve Everything	10
5	4. Indexing 1	11
5.1	4.1 Inverted Index	11
5.2	4.2 Dictionary and Postings	11
5.3	4.3 Index Construction	11
5.4	4.4 Boolean Retrieval	11
5.5	4.5 Phrase and Proximity Search	11
5.6	4.6 ASR Non-Deterministic Positional Index	12
6	5. Indexing 2: Compression and Wildcards	13
6.1	5.1 Delta Encoding	13
6.2	5.2 V-byte Encoding	13
6.3	5.3 Compression Tradeoff	13

6.4	5.4 Dictionary Data Structures	13
6.5	5.5 Permuterm Index	13
6.6	5.6 Character n-gram Index	14
6.7	5.7 Extent Index	14
7	6. Ranked Retrieval 1: TF-IDF and Vector Space Model	15
7.1	6.1 Ranked vs Boolean Retrieval	15
7.2	6.2 Jaccard Coefficient	15
7.3	6.3 TF, DF, CF, IDF	15
7.4	6.4 TF-IDF	15
7.5	6.5 Query-Document Score	16
7.6	6.6 Log Base and Ranking	16
7.7	6.7 Vector Space Model	16
7.8	6.8 Cosine Similarity	16
7.9	6.9 SMART Notation	16
8	7. Ranked Retrieval 2: Probabilistic Models and Language Models	17
8.1	7.1 Probability Ranking Principle	17
8.2	7.2 BM25	17
8.3	7.3 Query Likelihood Model	17
8.4	7.4 MLE and Zero Probability	17
8.5	7.5 Smoothing	18
8.6	7.6 Smoothing Calculation	18
8.7	7.7 Document Prior	18
9	8. IR Evaluation 1: Core Metrics	19
9.1	8.1 Cranfield Paradigm	19
9.2	8.2 Precision and Recall	19
9.3	8.3 F1 and F-beta	19
9.4	8.4 P@k and R@k	19
9.5	8.5 R-precision	20
9.6	8.6 Average Precision and MAP	20
9.7	8.7 MRR	20
9.8	8.8 DCG and nDCG	21
9.9	8.9 Custom Discount	21
10	9. IR Evaluation 2: Judgments, Logs, and Significance	22
10.1	9.1 Pooling	22
10.2	9.2 Relevance Assessors and Kappa	22
10.3	9.3 Web Search Evaluation	22
10.4	9.4 Clickthrough Data	22
10.5	9.5 A/B Testing	22
10.6	9.6 Significance Testing	22
11	10. Query Expansion and Feedback	23
11.1	10.1 Query Expansion	23
11.2	10.2 Thesaurus-Based QE	23
11.3	10.3 Explicit Relevance Feedback	23
11.4	10.4 Implicit Feedback	23
11.5	10.5 Rocchio Algorithm	23
11.6	10.6 Pseudo Relevance Feedback	24
11.7	10.7 Query Logs and Similar Queries	24
12	11. Comparing Corpora and Word-Class Association	25
12.1	11.1 Content Analysis	25
12.2	11.2 Automated Content Analysis	25

12.3	11.3	Mutual Information	25
12.4	11.4	Contingency Table	25
12.5	11.5	Why Frequency Alone Is Not MI	25
12.6	11.6	Chi-Squared	26
12.7	11.7	Lexicon / Dictionary Approach	26
13	12.	Topic Modelling	27
13.1	12.1	Unigram Model	27
13.2	12.2	Mixture of Unigrams	27
13.3	12.3	pLSI	27
13.4	12.4	LDA	27
13.5	12.5	Latent and Observable Variables	27
13.6	12.6	LDA Assumptions and Unrealism	28
13.7	12.7	Gibbs Sampling	28
13.8	12.8	Topic Labels	28
14	13.	Web Graph and PageRank	29
14.1	13.1	Web Graph	29
14.2	13.2	PageRank Meaning	29
14.3	13.3	PageRank Update Formula	29
14.4	13.4	Teleportation	29
14.5	13.5	PageRank without Teleportation	29
14.6	13.6	Anchor Text	30
14.7	13.7	Link Spam	30
15	14.	Web Search and Crawling	31
15.1	14.1	Web Search Components	31
15.2	14.2	User Needs	31
15.3	14.3	SEO and Spam	31
15.4	14.4	Duplicate and Near-Duplicate Detection	31
15.5	14.5	Shingles and MinHash	31
15.6	14.6	Web Crawling	32
15.7	14.7	Polite Crawler	32
15.8	14.8	URL Frontier and Back Queues	32
15.9	14.9	Freshness and Hacked Pages	32
16	15.	Text Classification	33
16.1	15.1	Classification Types	33
16.2	15.2	Rule-Based vs Supervised Classification	33
16.3	15.3	Feature Extraction	33
16.4	15.4	Feature Selection	33
16.5	15.5	Feature Scaling	33
16.6	15.6	Train / Development / Test	33
16.7	15.7	Evaluation under Imbalance	34
16.8	15.8	Bias and Fairness	34
16.9	15.9	Error Analysis	34
16.10	15.10	Zero-Shot and Transfer Learning	34
17	16.	Modern IR, LLMs, and RAG	35
17.1	16.1	LLMs vs Search Engines	35
17.2	16.2	Freshness and Attribution	35
17.3	16.3	Dense Retrieval	35
17.4	16.4	RAG	35
17.5	16.5	Query and Document Encoders	35
17.6	16.6	Vector Database Contents	36
17.7	16.7	Privacy-Preserving RAG	36

18 17. Past-Paper Selection Logic	37
18.1 17.1 The Pattern Behind 2023 Q1	37
18.2 17.2 Calculation Questions Pool	37
18.3 17.3 Short-Answer Template	37
18.4 17.4 Compare Question Template	37
18.5 17.5 Calculation Answer Template	38
19 Final Revision Checklist	38

1 How to Use This Guide

这份 handout 把 TTDS 的考试知识池拆成 Topic x atomic knowledge point pairs。Past papers 的 short-answer 题通常不是凭空出题，而是从这些 pairs 中抽取若干个，再包装成 define / explain why / compare / calculate / give examples / discuss failure cases。

建议你复习每个 knowledge point 时都能说出四件事：

1. Definition: 这个概念是什么。
2. Why it matters: 它在 IR / text analysis 系统里解决什么问题。
3. Exam hook: 过去或可能如何出题。
4. Pitfall: 最容易写错或算错的地方。

标记说明：

- [Past]: 在 2023-2025 past papers 中已经直接或近似出现。
- [Calc]: 可能要求手算。
- [Compare]: 常以比较题出现。

2 1. IR Basics and Definitions

2.1 1.1 Information Retrieval

Pair: IR Basics -> Information Retrieval definition

Information Retrieval (IR) 是从 large document collection 中, 根据 user 的 information need 返回 useful documents 的技术。它和简单字符串查找不同, 因为用户的 query 往往只是 information need 的不完美表达, 而 documents 是否 relevant 也常常带有主观性。

考试短答可以写:

IR retrieves documents/items from an unstructured or semi-structured collection in response to a user information need, usually ranking results by estimated relevance.

常见坑: 不要把 IR 说成 database lookup。Database 更接近 exact match; IR 更强调 ranking、uncertainty、relevance。

2.2 1.2 Query, Document, Information Need

Pair: IR Basics -> Query/document/information need

Information need 是用户真正想解决的问题; query 是用户输入给系统的表达; document 是被检索的 item, 可以是网页、tweet、论文、代码文件、图片 caption 或音频 transcript。

同一个 information need 可以有多个 query, 例如 “cheap flights to Paris” 和 “budget airline Paris June”。同一个 query 也可能对应多个 intent, 例如 “apple” 可以是 fruit、company 或 record label。

考试钩子: 如果问 query formulation 为什么难, 要答: 用户 query 短、含糊、有 synonymy/polysemy, 且不一定包含 relevant documents 的 exact terms。

2.3 1.3 Effectiveness and Efficiency

Pair: IR Basics -> Effectiveness vs efficiency

Effectiveness 衡量结果质量, 例如 precision、recall、nDCG。Efficiency 衡量系统资源, 例如 latency、throughput、memory、index size。

这两者经常 trade off。更复杂的 ranking model 可能提升 effectiveness, 但增加 query-time latency; 更强 compression 可以减少 I/O, 但需要 decompression CPU。

考试模板:

Effectiveness asks whether the system returns the right documents; efficiency asks whether it can do so fast and cheaply enough at scale.

2.4 1.4 Relevance

Pair: IR Basics -> Relevance

Relevance 是 document 是否满足 query/information need。它不是纯文本相似度: 一个 document 可能 share many words with the query 但不回答问题; 另一个 document 可能用 synonyms 表达却 relevant。

Relevance 的困难来自:

- topical relevance: 主题是否相关。
- user/context relevance: 是否满足当前用户和任务。
- graded relevance: 不是只有 relevant/non-relevant, 而是 highly/partially/not relevant。

常见坑: 不要只写 “same words = relevant”。IR 中这只是 bag-of-words 的近似假设。

2.5 1.5 Synonymy and Polysemy

Pair: IR Basics -> Vocabulary mismatch

Synonymy 是不同词表达相近意思，例如 “car” 和 “automobile”。Polysemy 是同一个词有多个意思，例如 “bank” 可以是 river bank 或 financial bank。

Synonymy 会降低 recall，因为 relevant documents 可能不用 query term。Polysemy 会降低 precision，因为 matching term 不一定 matching meaning。

关联考点：query expansion、dense retrieval、BERT/transformer reranking 都试图缓解 vocabulary mismatch。

2.6 1.6 Bag-of-Words

Pair: IR Basics -> Bag-of-Words representation

Bag-of-Words (BOW) 把 document 表示成 terms 的 multiset 或 vector，忽略大部分 word order。它简单、高效，适合 inverted index 和 sparse vector scoring。

优点：scalable、interpretable、works surprisingly well for topical relevance。缺点：丢失 syntax、negation、phrase meaning 和 long-range context。

考试模板：

BOW assumes that documents with similar term distributions are likely to be relevant to similar information needs, but it loses word order and deeper semantics.

2.7 1.7 IR Pipeline and RAG

Pair: IR Basics -> IR architecture and RAG

典型 IR 系统分成 offline indexing 和 online search。Offline 阶段做 preprocessing、index construction；online 阶段处理 query、retrieve candidates、rank results、log interactions。

RAG (Retrieval-Augmented Generation) 复用了这个架构：retriever 从 non-parametric memory 中找 evidence，generator 用 LLM 生成 answer。这里 IR 是 LLM 的 grounding layer。

Past paper 里 LLM vs search、RAG coding assistant、document encoder update cost 都属于这个区域。

3 2. Text Laws

3.1 2.1 Zipf's Law

Pair: Text Laws → Zipf's law

Zipf's law 描述 rank-frequency distribution: 第 r 高频的 term, 其 frequency roughly 与 $1/r$ 成反比。

常见形式:

$$f(r) \approx \frac{c}{r^\alpha}$$

在简化版中 $\alpha \approx 1$, 所以 $f(r) \approx c/r$ 。log-log 图上通常接近一条负斜率直线。

考试钩子: 如果让你 plot rank-frequency curve, 要用 x 轴 rank、y 轴 frequency, 两者取 log; 参数 c 可通过 fitting 或 $rf(r)$ 的平均/中位数估计。

3.2 2.2 Zipf and Stop Words

Pair: Text Laws → High-frequency terms

Zipf's law 解释了为什么少数 stop words 占据大量 token occurrences。比如 “the”, “of”, “and” 的 frequency 很高, 但通常 topic specificity 很低。

对 IR 的影响: 高频词 postings list 极长, query processing 代价高; 它们的 IDF 很低, 对 ranking 贡献小。

常见坑: 高频不等于无用。短语、实体名、歌曲名、negation 中 stop words 可能非常重要。

3.3 2.3 Long Tail and Rare Terms

Pair: Text Laws → Rare terms and long tail

大量 terms 只出现一次或很少出现, 这就是 vocabulary long tail。Rare terms 往往有较高 IDF, 对 precise matching 有用, 但也可能是 typo、noise 或 one-off entities。

对 indexing 的影响: dictionary 很大, postings lists 大量很短; compression 和 efficient dictionary lookup 变得重要。

3.4 2.4 Heaps' Law

Pair: Text Laws → Vocabulary growth [Past] [Calc]

Heaps' law 描述 corpus token 数 N 增长时, vocabulary size V 如何增长:

$$V = kN^b$$

取 log 后:

$$\log V = \log k + b \log N$$

所以可以在 log-log 图上用 linear regression 估计 k 和 b 。 b 通常在 0.4 到 0.6 左右, 但不同 corpus 可能不同。

Past paper 2025 问过: closed-domain collection 和 general-domain collection 哪个 b 更小。答案是 closed-domain 通常更小, 因为 vocabulary 更快复用、增长更慢。

3.5 2.5 Document Order in Vocabulary Growth

Pair: Text Laws -> Order effect

如果 collection 很大，整体 Heaps parameters 应该比较稳定；但早期 curve 会受 document order 影响。先处理 diverse topics, vocabulary 会快速增长；先处理同一主题 documents, 增长会慢。

考试答法：

The order should not change the final vocabulary size, but it can affect the early shape of the growth curve and therefore parameter estimates on small prefixes.

3.6 2.6 Clumping / Contagion

Pair: Text Laws -> Clumping

Clumping 指某个 term 一旦在 document 中出现，就更可能在同一 document 中再次出现。Text terms 不像 independent random events；它们和 topic strongly correlated。

对 retrieval 的启发：term frequency 有意义，因为 repeated occurrence often signals topical focus。对 compression 的启发：postings gaps 和 positions 往往有 patterns，可以被压缩。

4 3. Preprocessing

4.1 3.1 Preprocessing Pipeline

Pair: Preprocessing -> Pipeline

Text preprocessing 把 raw text 转成 indexable terms。常见步骤包括 tokenisation、case folding、normalisation、stopping、stemming/lemmatisation。

最重要原则：document 和 query 必须使用 compatible preprocessing。否则 query term 和 index term 不在同一个 representation space。

4.2 3.2 Tokenisation

Pair: Preprocessing -> Tokenisation [Past]

Tokenisation 是把 text 切成 tokens。简单英语文本可以按 whitespace/punctuation 切，但真实文本中有 hyphenated words、URLs、hashtags、emojis、numbers、abbreviations。

例子：

- COVID-19: 保留整体还是拆成 covid 和 19?
- U.S.A.: period 是缩写的一部分还是句号?
- don't: 保留 don't, 还是拆成 do 和 n't?

考试答法要给 benefit 和 drawback。规则越激进，vocabulary 越小，但越可能破坏 meaning。

4.3 3.3 Stopword Removal

Pair: Preprocessing -> Stopping [Past]

Stop words 是高频 function words，如 the, of, is。Removing stopwords 可以减小 index、减少 noise、加速 query processing。

风险：有些 query 的 meaning 依赖 stop words，例如 “to be or not to be”，“The Who”，“The Office”，“No Country for Old Men”。现代 web search 通常不简单删除所有 stop words，因为用户 query 短，任何词都可能有用。

考试模板：

Stopword removal improves efficiency and sometimes precision, but it can harm phrase queries, named entities, negation, and exact-title search.

4.4 3.4 Removing URLs or Non-Alphanumeric Characters

Pair: Preprocessing -> URL/non-alphanumeric removal [Past]

删除 URLs、punctuation、symbols 可以减少 sparse noise。比如 tweets 里的 random URLs 会制造大量 rare tokens。

但 URLs 也可能包含重要 evidence：domain name 表示 source credibility，path 中可能有 topic words，hashtags/emojis/punctuation 可能携带 sentiment。

Past paper 2024/2025 都考过这种 “explain step + advantage + drawback + example” 格式。

4.5 3.5 Case Folding and Normalisation

Pair: Preprocessing -> Case folding / normalisation

Case folding 把 Apple 和 apple 都转成 apple，提升 matching recall。风险是丢失 proper noun 和 common noun 的区别。

Normalisation 还包括 accent removal、spelling variants、equivalence classes, 例如 colour/color 或 cafe/café。

核心判断: normalisation 应该保留 task-relevant distinctions, 同时合并不重要的 surface variants。

4.6 3.6 Stemming

Pair: Preprocessing -> Stemming [Past]

Stemming 把 inflected/derived words 归并到 stem, 例如 connect, connected, connection 可能都变成 connect。它提升 recall, 因为不同 morphological variants 可以 match。

风险:

- Overstemming: unrelated words 被错误合并, precision 下降。
- Understemming: related words 没被合并, recall 下降。

Porter stemmer aggressive, Krovetz stemmer conservative, 后者更倾向产生 valid words。

4.7 3.7 Language-Specific Preprocessing

Pair: Preprocessing -> Multilingual preprocessing

不同语言不能照搬英语规则。Chinese/Japanese 没有空格, 需要 word segmentation; German compounds 可能需要 compound splitting; Arabic 等语言 morphology 复杂, stemming/lemmatisation 更重要。

考试如果问 “why BOW needs adaptation for Chinese/Japanese/German”, 核心就是 token boundary 和 morphology 不同。

4.8 3.8 Preprocessing Cannot Solve Everything

Pair: Preprocessing -> Limits

Preprocessing 主要处理 surface form mismatch。它不能充分解决 semantic mismatch, 例如 “heart attack” 和 “myocardial infarction”。这种问题通常需要 query expansion、thesaurus、embeddings 或 neural retrieval。

5 4. Indexing 1

5.1 4.1 Inverted Index

Pair: Indexing → Inverted index

Inverted index 从 term 映射到 postings list。每个 postings list 包含该 term 的 documents，可能还包含 term frequency、positions、fields。

结构：

$$\text{term} \rightarrow [(d_1, tf_1, positions_1), (d_2, tf_2, positions_2), \dots]$$

它是 sparse term-document matrix 的高效表示，避免扫描整个 collection。

5.2 4.2 Dictionary and Postings

Pair: Indexing → Dictionary/postings

Dictionary 存 terms 及其 metadata，例如 df、pointer to postings。Posting 是一个 term 在某个 document 中的信息。Postings list 是某 term 的所有 postings。

考试中如果问 df/cf：

- df_t : number of documents containing term t 。
- cf_t : total occurrences of term t in the collection。

df 更常用于 IDF；cf 更常用于 collection language model。

5.3 4.3 Index Construction

Pair: Indexing → Construction workflow

基本步骤：

1. Parse documents and generate term-doc-position triples。
2. Sort triples by term then docID。
3. Merge into postings lists。
4. Store dictionary and compressed postings。

常见短答：为什么 indexing 是 offline？因为 construction 成本高，但可以提前完成；search 是 online，需要低 latency。

5.4 4.4 Boolean Retrieval

Pair: Indexing → Boolean retrieval

Boolean retrieval 对 query 做 exact logical matching，例如 ink AND wink。AND 对应 postings intersection，OR 对应 union，NOT 对应 set difference。

优点：precise、interpretable、适合法律/专利等 expert search。缺点：无 ranking，用户难写 query，结果集可能太大或太小。

5.5 4.5 Phrase and Proximity Search

Pair: Indexing → Positional index [Past]

Phrase search 不要求 terms 出现，还要求 positions 连续。例如 "pink ink" 需要 pink 在位置 p ，ink 在位置 $p + 1$ 。

Proximity search 放宽为距离约束，例如 data NEAR/3 science。

位置索引代价: postings 更大, 但能支持 phrase/proximity queries, 提高 precision。

5.6 4.6 ASR Non-Deterministic Positional Index

Pair: Indexing → Probabilistic postings for ASR [Past]

2025 Q2 给了 ASR noisy transcripts: 每个 audio segment 有多个 candidate words 和 probabilities。此时 positional index 可以在每个 position 存多个 term-probability pairs。

posting 可能长这样:

$$\text{jobs} \rightarrow [(d_1, \text{position} = 2, \text{prob} = 0.35)]$$

这种系统通常提升 recall, 因为正确词即使不是 top-1 也可能被检索到; 但可能降低 precision, 因为额外 candidates 引入 false matches。Probability-aware scoring 可以缓解这个问题。

6 5. Indexing 2: Compression and Wildcards

6.1 5.1 Delta Encoding

Pair: Compression → Delta/gap encoding [Past] [Calc]

Postings list 中 docIDs 通常递增。Delta encoding 不存原始 docID，而存相邻 docID 的 gap。

例子：

$$[18, 578067, 579224] \rightarrow [18, 578049, 1157]$$

优点：gaps 通常比 docIDs 小，更容易压缩。

常见坑：v-byte 解出来的常常是 gaps，不是最终 postings list；要 cumulative sum 才能还原。

6.2 5.2 V-byte Encoding

Pair: Compression → Variable byte encoding [Past] [Calc]

V-byte encoding 用若干 bytes 表示一个 integer，每个 byte 通常 7 bits 存 data，1 bit 表示是否结束。讲义/题目会给定 convention，例如 high bit 1 means terminates。

答题步骤：

1. 按 termination bit 切分 byte groups。
2. 去掉 control bit，拼接 7-bit payload。
3. 转成 decimal integers。
4. 如果是 delta-encoded postings，再做 cumulative sum。

6.3 5.3 Compression Tradeoff

Pair: Compression → Space-time tradeoff

Compression 减少 disk/memory I/O，可能让 query processing 更快，因为 bottleneck 常是读取 postings。但 decompression 需要 CPU。更高压缩率通常意味着更复杂 decoding。

考试模板：

Compression helps because smaller postings lists reduce storage and I/O, but the system pays extra CPU cost to decode them.

6.4 5.4 Dictionary Data Structures

Pair: Indexing → Dictionary structures

Hash table lookup fast for exact term lookup，但不支持 prefix/range traversal。Tree/B-tree 支持 sorted traversal 和 prefix search，适合 wildcard/prefix queries，但 lookup 常数更高。

如果问 B-tree 为什么常用于 index dictionary：因为它 disk/cache friendly，支持 ordered lookup 和 range scan。

6.5 5.5 Permuterm Index

Pair: Wildcard → Permuterm index [Past]

Permuterm index 给每个 term 加 \$，存所有 rotations。查询 wildcard 时旋转到 * 在末尾，再做 prefix lookup。

例子：hello\$ 的 rotations 包括：

hello\$, ello\$h, llo\$he, lo\$hel, o\$hell, \$hello

查询 `blo*k`: 先写成 `blo*k$`, 旋转为 `k$blo*`, 查 prefix `k$blo`。

主要缺点: dictionary size blow-up。长度为 m 的 term 会产生 $m + 1$ rotations。

6.6 5.6 Character n-gram Index

Pair: Wildcard $\rightarrow$ Character n-gram index

Character n-gram index 把 term 拆成 character bigrams/trigrams。用 `$` 表示边界, 例如 `cat` 的 bigrams 可为 `$c`, `ca`, `at`, `t$`。

Wildcard query 先转成 n-gram constraints, 再 intersect candidate terms, 最后 post-filter。Post-filter 必须做, 因为 n-gram match 只是 necessary condition, 不一定满足原 wildcard pattern。

6.7 5.7 Extent Index

Pair: Indexing $\rightarrow$ Structured documents

Extent index 存 document 的结构 span, 例如 title、heading、body 的 start/end positions。它支持 field-specific queries, 如 `title:retrieval`。

优点: 可以利用 document structure, 提高 precision; 代价是 index 更复杂。

7 6. Ranked Retrieval 1: TF-IDF and Vector Space Model

7.1 6.1 Ranked vs Boolean Retrieval

Pair: Ranked Retrieval → Boolean vs ranked

Boolean retrieval 只返回 match/non-match。Ranked retrieval 给每个 document 一个 score，并按 score 降序返回 top-k。

Web search 更适合 ranked retrieval，因为用户不愿写复杂 Boolean query，也通常只看前几个结果。

7.2 6.2 Jaccard Coefficient

Pair: Ranked Retrieval → Jaccard [Past] [Calc]

Jaccard coefficient 衡量两个 sets 的 overlap:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

用于 IR 时，可把 query terms 和 document terms 看成 sets。缺点：忽略 term frequency，也忽略 rare terms 的重要性。一个 rare query term 和一个 stopword 被同等对待。

7.3 6.3 TF, DF, CF, IDF

Pair: Ranked Retrieval → Term statistics [Past] [Calc]

核心统计：

- $tf_{t,d}$: term t 在 document d 中出现次数。
- df_t : 有多少 documents 包含 t 。
- cf_t : t 在整个 collection 出现总次数。
- N : collection 中 document 总数。

IDF 衡量 term 的 discriminativeness:

$$idf_t = \log \frac{N}{df_t}$$

如果 $df_t = N$ ，则 $idf_t = 0$ ，说明 term 出现在所有 documents 中，区分能力很弱。

7.4 6.4 TF-IDF

Pair: Ranked Retrieval → TF-IDF scoring [Past] [Calc]

TF-IDF 的直觉：term 在 document 内频繁出现说明 document 关注它；term 在 collection 中少见说明它更有区分力。

常见权重：

$$w_{t,d} = (1 + \log tf_{t,d}) \cdot \log \frac{N}{df_t}$$

如果题目说 “do not squash tf”，就不要用 $1 + \log tf$ ，而用 raw tf 。2024 Q3(d) 就明确要求按 raw tf-idf weighted sum score 排序。

7.5 6.5 Query-Document Score

Pair: Ranked Retrieval → Weighted sum

简单 TF-IDF ranking 可写成：

$$\text{score}(q, d) = \sum_{t \in q} tf_{t,d} \cdot idf_t$$

如果 query term 不在 document 中，对 score 的贡献为 0。考试手算时只需要看 query terms，不需要为所有 vocabulary terms 建完整向量。

7.6 6.6 Log Base and Ranking

Pair: Ranked Retrieval → Log base in TF-IDF [Past]

2025 Q1(c) 问过把 $\log_{10}$ 换成 $\log_2$ 的影响。关键恒等式：

$$\log_2 x = \frac{\log_{10} x}{\log_{10} 2}$$

所以换 base 会把 log value 乘上 positive constant。若所有 documents 的 score 被同样 scaling，ranking 通常不变。但如果和其他未缩放 features 混合，或出现 ties，可能影响排序。

7.7 6.7 Vector Space Model

Pair: Ranked Retrieval → Vector Space Model

VSM 把 query 和 document 表示成同一 vocabulary space 中的 weighted vectors。每一维对应一个 term，值通常是 TF-IDF weight。

直觉：documents 和 query 的方向越相似，越 relevant。

7.8 6.8 Cosine Similarity

Pair: Ranked Retrieval → Cosine similarity

Cosine similarity 衡量两个向量夹角，而不是长度：

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

如果 vectors 已经 L2-normalized：

$$\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d}$$

考试常见解释：length normalization 防止长文档因为词多而天然拿高分。

7.9 6.9 SMART Notation

Pair: Ranked Retrieval → SMART weighting

SMART notation 用三个字母描述 tf、idf、normalization。例如 `lnc.ltc` 表示 document 端和 query 端使用不同 weighting。你不一定要背所有 variants，但要知道它是在系统化表示 weighting scheme。

8 7. Ranked Retrieval 2: Probabilistic Models and Language Models

8.1 7.1 Probability Ranking Principle

Pair: Probabilistic IR → PRP

Probability Ranking Principle (PRP) 说，如果能准确估计每个 document 对 query 的 relevance probability，那么按这个 probability 降序排列是 optimal 的。

考试模板：

PRP ranks documents by decreasing estimated probability of relevance to the query, assuming estimates are accurate and user utility follows relevance.

8.2 7.2 BM25

Pair: Probabilistic IR → BM25

BM25 可以看作 TF-IDF 的强 baseline 改进版。它包含 IDF、tf saturation 和 document length normalization。

核心直觉：

- term 出现越多，score 越高，但 marginal gain saturates。
- rare query terms 权重大。
- 太长的 document 不应仅因词多而占便宜。

考试短答不一定要求公式，但要能说出这三件事。

8.3 7.3 Query Likelihood Model

Pair: Language Model for IR → Query likelihood [Past]

Language Model approach 把每个 document 看成一个语言模型 M_d ，ranking 依据是 document model 生成 query 的概率：

$$\text{score}(q, d) = P(q \mid M_d)$$

在 unigram independence assumption 下：

$$P(q \mid M_d) = \prod_{t \in q} P(t \mid M_d)$$

实际计算常用 log probability，避免 underflow：

$$\log P(q \mid M_d) = \sum_{t \in q} \log P(t \mid M_d)$$

8.4 7.4 MLE and Zero Probability

Pair: Language Model for IR → MLE zero problem [Past]

Document language model 的 MLE：

$$P_{MLE}(t \mid d) = \frac{tf_{t,d}}{|d|}$$

如果 query term 没在 document 里，则 $P(t \mid d) = 0$ ，整个 $P(q \mid d)$ 变成 0。这太 harsh，因为 document 可能仍然 relevant，只是没出现某个 exact term。

这就是 smoothing 的动机。

8.5 7.5 Smoothing

Pair: Language Model for IR → Smoothing [Past]

Smoothing 把 document model 和 collection model 混合，给 unseen terms 非零概率，也让短 documents 的估计更稳。

Jelinek-Mercer smoothing:

$$P(t | d) = \lambda P_{MLE}(t | d) + (1 - \lambda)P(t | C)$$

其中 $P(t | C) = cf_t / |C|$ 。高 λ 更相信 document 本身；低 λ 更相信 collection background。

8.6 7.6 Smoothing Calculation

Pair: Language Model for IR → Jelinek-Mercer calculation [Past] [Calc]

2024 Q3(c) 这类题会给 document frequency/counts，让你算 $P(fat | d_3)$ 。步骤：

1. 算 $P_{MLE}(fat | d_3) = tf(fat, d_3) / |d_3|$ 。
2. 算 $P(fat | C) = cf(fat) / |C|$ 。
3. 代入 JM smoothing。

常见坑： λ 的权重方向要按题目公式；有的课程用 λ 给 document model，有的给 collection model，必须看题目。

8.7 7.7 Document Prior

Pair: Language Model for IR → Document prior

如果从 Bayes 角度：

$$P(d | q) \propto P(q | d)P(d)$$

多数简单模型假设 $P(d)$ uniform。但 web search 可把 PageRank、freshness、popularity、authority 等作为 document prior。

9 8. IR Evaluation 1: Core Metrics

9.1 8.1 Cranfield Paradigm

Pair: Evaluation -> Cranfield test collection [Past]

Cranfield paradigm 用固定 test collection 做可重复的 offline evaluation。核心组件:

1. document collection
2. topics / information needs / queries
3. relevance judgments / qrels
4. evaluation measure

2023 Q1(a) 直接考过 components。答题不要漏 qrels。

9.2 8.2 Precision and Recall

Pair: Evaluation -> Precision/Recall [Past] [Calc]

Precision 衡量 retrieved results 中有多少 relevant:

$$Precision = \frac{TP}{TP + FP}$$

Recall 衡量所有 relevant 中找回多少:

$$Recall = \frac{TP}{TP + FN}$$

IR 中 precision 更关注用户看到的结果质量, recall 更关注 relevant documents 是否被找全。

9.3 8.3 F1 and F-beta

Pair: Evaluation -> F-measure [Past] [Calc]

F1 是 precision 和 recall 的 harmonic mean:

$$F_1 = \frac{2PR}{P + R}$$

如果 recall 更重要, 可用 F_β :

$$F_\beta = (1 + \beta^2) \frac{PR}{\beta^2 P + R}$$

$\beta > 1$ 时 recall 权重更大。2023 misinformation classifier 适合强调 recall。

9.4 8.4 P@k and R@k

Pair: Evaluation -> Top-k metrics [Past] [Calc]

P@k 只看 ranked list 前 k 个结果:

$$P@k = \frac{\#relevant\ in\ top\ k}{k}$$

R@k:

$$R@k = \frac{\#relevant\ in\ top\ k}{\#relevant\ in\ collection}$$

2024 Q2 考过 $R@6$ 。常见坑：denominator 不同， $P@k$ 除以 k ， $R@k$ 除以 total relevant。

9.5 8.5 R-precision

Pair: Evaluation -> R-precision [Past] [Calc]

R-precision 是 $P@R$ ，其中 R 是该 query 的 total relevant documents 数。

$$R\text{-precision} = \frac{\#relevant\ in\ top\ R}{R}$$

2023 Q4(b) 要对多个 queries 算 R-precision。

9.6 8.6 Average Precision and MAP

Pair: Evaluation -> AP/MAP [Past] [Calc]

Average Precision (AP) 在每个 relevant document 出现的位置计算 precision，然后平均：

$$AP = \frac{1}{R} \sum_{k=1}^n P@k \cdot rel_k$$

其中 $rel_k = 1$ 表示 rank k 是 relevant，否则为 0。MAP 是多个 queries 的 AP 平均：

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q)$$

常见坑：如果只是一条 query，MAP equals AP。

9.7 8.7 MRR

Pair: Evaluation -> Mean Reciprocal Rank [Past] [Calc]

MRR 只关心第一个 relevant result 的 rank：

$$RR = \frac{1}{rank_{first\ relevant}}$$

多 queries 时：

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{rank_q}$$

2025 Q1(e) 规定只有 relevance 3 才 considered relevant，所以第一个 relevance-3 在 rank 3， $RR = 1/3$ 。

9.8 8.8 DCG and nDCG

Pair: Evaluation -> Graded relevance and nDCG [Past] [Calc]

DCG 用 graded relevance 和 rank discount:

$$DCG@k = \sum_{i=1}^k \frac{gain_i}{\log_2(i+1)}$$

nDCG 用 ideal DCG normalization:

$$nDCG@k = \frac{DCG@k}{IDCG@k}$$

关键: IDCG 是把 gains 按最优顺序排列后得到的 DCG。

9.9 8.9 Custom Discount

Pair: Evaluation -> Custom nDCG variants [Past] [Calc]

2023 Q4(c) 考过 linear discount 的 modified nDCG。遇到自定义 metric 时, 不要套标准公式; 要先按题意定义 discount weight, 再算 DCG 和 ideal DCG。

比如 rank 1 weight = 1, rank 10 weight = 0.1, 可定义:

$$w_i = 1 - 0.1(i - 1), \quad 1 \leq i \leq 10$$

然后:

$$DCG'@10 = \sum_{i=1}^{10} gain_i w_i$$

10 9. IR Evaluation 2: Judgments, Logs, and Significance

10.1 9.1 Pooling

Pair: Evaluation -> Pooling

Pooling 是构建 relevance judgments 的方法：把多个 systems 的 top results 合并成 pool，让 assessors 判断这些 documents。它省成本，但可能漏掉没有系统 retrieve 到的 relevant documents。

考试如果问 limitation: pooling depends on systems included; judgments may be incomplete。

10.2 9.2 Relevance Assessors and Kappa

Pair: Evaluation -> Assessor agreement

Relevance 有主观性，所以可能需要多个 assessors。Cohen' s kappa 衡量 agreement beyond chance:

$$\kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

$P(A)$ 是 observed agreement, $P(E)$ 是 expected agreement by chance。 $\kappa = 1$ 完全一致, $\kappa = 0$ 仅等于随机一致。

10.3 9.3 Web Search Evaluation

Pair: Evaluation -> Web search evaluation

Web search 中 recall 难测，因为 web-scale collection 中 total relevant documents 不可知。常用 top-heavy metrics, 如 P@k、nDCG@k、click metrics、A/B testing。

短答重点: web users 只看 top few results, 所以 top ranks 的质量特别重要。

10.4 9.4 Clickthrough Data

Pair: Evaluation -> Clickthrough as implicit feedback [Past]

Clickthrough data 可以推断 user preference、query intent、result attractiveness。但 clicks 有 bias: position bias、snippet bias、presentation bias。排名靠前的 result 更容易被点，不一定更 relevant。

考试模板:

Clicks are useful implicit feedback signals, but they are noisy and biased by rank position and presentation, so they require careful debiasing or aggregation.

10.5 9.5 A/B Testing

Pair: Evaluation -> Online experiments

A/B testing 把用户随机分到 control 和 treatment, 比较 clicks、satisfaction、latency、success rate 等指标。大型 search engines 重视它，因为 offline metrics 不一定完全代表真实用户行为。

10.6 9.6 Significance Testing

Pair: Evaluation -> Statistical significance

系统平均 MAP 更高不一定表示真实更好，可能只是 query sample noise。Significance test 的 null hypothesis 通常是 two systems have no real difference。

常用 per-query scores 做 paired test, 例如 paired t-test 或 Wilcoxon signed-rank test。AP 是 per-query score; MAP 是 aggregate score, 所以做显著性检验时用每个 query 的 AP。

11 10. Query Expansion and Feedback

11.1 10.1 Query Expansion

Pair: Query Expansion → Vocabulary mismatch

Query expansion (QE) 向 query 添加 related terms, 以解决 user query 和 relevant documents 用词不一致的问题。

风险: query drift。扩展词如果偏离原 intent, 会提升 noise、降低 precision。

11.2 10.2 Thesaurus-Based QE

Pair: Query Expansion → Thesaurus

Thesaurus 可以是 manual, 也可以从 co-occurrence 或 parallel corpora 自动构建。它把 query term 映射到 synonyms/related terms。

优点: 可解释。缺点: generic thesaurus 常不符合 domain/context; polysemy 会导致错误扩展。

11.3 10.3 Explicit Relevance Feedback

Pair: Feedback → Explicit relevance feedback [Past]

Explicit relevance feedback 让用户标记 documents relevant/non-relevant, 然后系统更新 query/model。理论上有用, 因为它直接从用户那里获得 relevance signals。

为什么商业 web search 少见? 因为用户不愿额外标注, interaction cost 高, web search 常是快速任务。

11.4 10.4 Implicit Feedback

Pair: Feedback → Implicit feedback [Past]

Implicit feedback 从用户行为推断 relevance, 例如 clicks、dwell time、scrolling、eye tracking、query reformulation。

Eye tracking 可以作为 attention signal, 但 noisy: 看得久可能表示感兴趣, 也可能是困惑或页面难懂。

11.5 10.5 Rocchio Algorithm

Pair: Feedback → Rocchio [Past] [Calc]

Rocchio 在 vector space 中移动 query: 靠近 relevant documents, 远离 non-relevant documents。

$$\vec{q}_m = \alpha \vec{q}_0 + \frac{\beta}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \frac{\gamma}{|D_{nr}|} \sum_{\vec{d}_j \in D_{nr}} \vec{d}_j$$

变量:

- $\vec{q}_0$: original query vector
- D_r : relevant documents
- D_{nr} : non-relevant documents
- α, β, γ : control original/relevant/non-relevant influence

2024 Q3(e) 考过用 tf-idf vectors 手算新 query。

11.6 10.6 Pseudo Relevance Feedback

Pair: Feedback -> Pseudo relevance feedback

PRF 假设 initial ranking 的 top-k documents relevant, 从中抽 top terms 扩展 query。它不需要用户标注, 适合自动化。

风险: 如果 top-k 结果本来错了, 会 query drift。评价 PRF 时不能用 test qrels 泄漏信息; 只能基于 retrieval results 自动扩展。

11.7 10.7 Query Logs and Similar Queries

Pair: Query Expansion -> Query logs [Past]

如果两个 queries 没有 shared words, 系统仍可通过 click-through overlap、reformulation sequences、dense embeddings 判断它们 similar。

Past paper 2023 Q1(i) 就考过这个点。答题可以写: shared clicked documents, query reformulation logs, embeddings/dense representations。

12 11. Comparing Corpora and Word-Class Association

12.1 11.1 Content Analysis

Pair: Comparing Corpora → Manual content analysis [Past]

Manual content analysis 的流程:

1. Define research question and coding categories.
2. Sample documents.
3. Create codebook.
4. Train annotators.
5. Code documents.
6. Measure agreement.
7. Resolve disagreements and analyse trends.

Past paper 2024 Q5 直接问过。

12.2 11.2 Automated Content Analysis

Pair: Comparing Corpora → Automated classification [Past]

当数据量大、categories 明确、training labels 足够、错误成本可接受时，适合用 text classifier 自动化。若任务很细腻、labels 主观、数据少或错误代价高，则不适合完全自动化。

考试要会给例子：大规模新闻主题分类适合；敏感法律判定或小样本细粒度 rhetorical coding 不适合。

12.3 11.3 Mutual Information

Pair: Comparing Corpora → Expected mutual information [Past] [Calc]

MI 衡量 term occurrence U 和 class C 的 association。若完全 independent，则 $I(U; C) = 0$ 。

二元 term-class 的公式:

$$I(U; C) = \sum_{u \in \{0,1\}} \sum_{c \in \{0,1\}} P(U = u, C = c) \log_2 \frac{P(U = u, C = c)}{P(U = u)P(C = c)}$$

Past papers 2023/2024/2025 都考过 contingency table + MI calculation。

12.4 11.4 Contingency Table

Pair: Comparing Corpora → $N_{11}/N_{10}/N_{01}/N_{00}$ [Past] [Calc]

对某 term 和某 class:

- N_{11} : term appears and document is in class.
- N_{10} : term appears and document is not in class.
- N_{01} : term absent and document is in class.
- N_{00} : term absent and document is not in class.

常见坑：不要把 N_{01} 和 N_{10} 反了；先画 2x2 表再代公式。

12.5 11.5 Why Frequency Alone Is Not MI

Pair: Comparing Corpora → Frequency vs association [Past]

某词在 class 中出现次数多，不一定 MI 更高。MI 关注的是 term occurrence 是否 disproportionately associated with the class。

如果一个词 everywhere frequent, 它可能在 target class 中频数也高, 但区分力不强。

2024 Q4(c) 问过 apple/llama 哪个在 Tech 中更多, 哪个 MI 更高, 并解释差异。

12.6 11.6 Chi-Squared

Pair: Comparing Corpora → Chi-squared

Chi-squared test 检验 observed counts 是否显著偏离 independence expectation。它和 MI 都能用于找 class-distinctive words 或 feature selection。

简单答法:

Chi-squared measures whether the association between a term and a class is stronger than expected under independence.

12.7 11.7 Lexicon / Dictionary Approach

Pair: Comparing Corpora → Lexicon method [Past]

Sentiment dictionary 或 LIWC-like lexicon 把 words 映射到 categories/scores。它适合可解释、低资源、无需训练数据的场景。

缺点: domain dependence、context ignorance、sarcasm/negation 处理差。2025 Q4(b) 问过什么时候用 sentiment dictionary instead of ML。

13 12. Topic Modelling

13.1 12.1 Unigram Model

Pair: Topic Modelling -> Unigram model [Past]

Unigram model 假设所有 words 都从同一个 global word distribution 独立生成。它忽略 documents 和 topics。

句子概率：

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

局限：不能表达不同 documents/topics 的词分布差异。

13.2 12.2 Mixture of Unigrams

Pair: Topic Modelling -> Mixture of unigrams [Past] [Compare]

Mixture of unigrams 假设每个 document 先选择一个 topic z ，然后 document 中所有 words 都从该 topic 的 word distribution 生成。

优点：比 unigram model 多了 document-level topic variation。缺点：一篇 document 只能有一个 topic，不适合混合主题文章。

13.3 12.3 pLSI

Pair: Topic Modelling -> pLSI [Past] [Compare]

pLSI 允许每个 document 有 topic mixture。生成 word 时，根据 document-specific topic distribution 选择 topic，再生成 word。

局限：参数和 training documents 绑定，对 unseen documents 不自然；相比 LDA 缺少 Dirichlet prior 的 generative story。

2023 Q6(b) 考过 pLSI 相比 LDA 的 limitation。

13.4 12.4 LDA

Pair: Topic Modelling -> LDA [Past]

Latent Dirichlet Allocation (LDA) 是生成式 topic model。每个 document 有 topic distribution θ_d ，每个 topic 有 word distribution ϕ_k 。

生成过程：

1. For each document, sample topic distribution θ_d from Dirichlet prior。
2. For each word token, sample topic z from θ_d 。
3. Sample observed word w from topic-word distribution ϕ_z 。

13.5 12.5 Latent and Observable Variables

Pair: Topic Modelling -> Latent variables [Past]

在 LDA 中 observable 是 documents 和 word tokens。Latent 是 topic assignments z 、document-topic distributions θ 、topic-word distributions ϕ 。

2025 Q4(a) 问过“latent”指什么。答题要明确：topic structure is not directly observed; only words are observed.

13.6 12.6 LDA Assumptions and Unrealism

Pair: Topic Modelling -> LDA assumptions [Past]

LDA 的 generative process 对现实语言很 simplified: 它把 words 当作 exchangeable bag-of-words, 忽略 syntax、word order、discourse coherence。

例子: 一句话 “The dog chased the cat” 和 “The cat chased the dog” 在 BOW/topic view 中非常相似, 但语义不同。

13.7 12.7 Gibbs Sampling

Pair: Topic Modelling -> Gibbs sampling [Past]

Gibbs sampling 是 approximate inference 方法。它反复给每个 word token resample topic assignment, 基于当前 document-topic counts 和 topic-word counts。

考试短答:

Remove the current token's topic assignment from count matrices, compute probabilities for each topic given remaining assignments, sample a new topic, then update counts.

常见坑: 要先 decrement 当前 token 的 counts, 否则它会影响自己的新 assignment。

13.8 12.8 Topic Labels

Pair: Topic Modelling -> Topic labels [Past]

LDA 输出的是 topic-word distributions, 不自动输出 human-readable labels。Topic labels 通常由 researcher 查看 top words 后人工命名。

所以如果问 topic labels 从哪里来, 答案不是 “LDA gives labels”, 而是 human interpretation。

14 13. Web Graph and PageRank

14.1 13.1 Web Graph

Pair: Web → Directed graph

Web 可以看成 directed graph: pages 是 nodes, hyperlinks 是 directed edges。Link analysis 利用网页之间的引用结构估计 importance。

Link assumption: 如果 page A links to page B, A 可能在某种意义上 endorse 或 reference B。

14.2 13.2 PageRank Meaning

Pair: Web → PageRank meaning [Past]

PageRank 是 random surfer 在长期随机浏览中位于某 page 的 stationary probability。它是 query-independent、link-based importance signal。

Past paper 2023 Q1(c) 直接问 “PageRank value represents what”。

14.3 13.3 PageRank Update Formula

Pair: Web → PageRank computation [Past] [Calc]

带 teleportation 的简化公式:

$$PR_{i+1}(x) = \frac{1 - \lambda}{N} + \lambda \sum_{y \rightarrow x} \frac{PR_i(y)}{L_{out}(y)}$$

变量:

- N : number of pages。
- λ : damping/follow-link probability。
- $L_{out}(y)$: page y 的 out-degree。

常见坑: 一个 page 的 PR 要平均分给它的 outlinks, 不是每条 outlink 都拿 full PR。

14.4 13.4 Teleportation

Pair: Web → Teleportation [Past]

Teleportation 让 random surfer 有一定概率跳到 random page。它解决 rank sinks、dead ends、disconnected components 等问题, 并帮助得到 robust unique stationary distribution。

2025 Q5(c) 问过为什么需要或不需要 teleportation。

14.5 13.5 PageRank without Teleportation

Pair: Web → PageRank without teleportation [Past] [Calc]

Without teleportation 时, iteration 只沿 links 传播 probability mass。若 graph 中有 sink/cycle, probability 可能集中在某些 component, ranking 不稳定或不反映 global importance。

考试计算时仍按:

$$PR_{i+1}(x) = \sum_{y \rightarrow x} \frac{PR_i(y)}{L_{out}(y)}$$

然后按每轮结果排序。

14.6 13.6 Anchor Text

Pair: Web -> Anchor text

Anchor text 是 hyperlink 上可见文字。它像 human-generated query expansion，因为别的 pages 用自己的词描述 target page。

用途：把 anchor text 加入 target page 的 index representation，帮助匹配 target page 自己没写出的 terms。

14.7 13.7 Link Spam

Pair: Web -> PageRank vulnerabilities

因为 ranking signals 会影响 traffic，网站会尝试 game algorithm。Link farms、comment spam、anchor text spam 都是 manipulating link/anchor signals 的方式。

rel=nofollow 是让 crawlers/ranking systems 不把某些 links 当作 endorsement 的机制。

15 14. Web Search and Crawling

15.1 14.1 Web Search Components

Pair: Web Search → Search engine architecture [Past]

Web search engine 主要组件:

- crawler/spider: fetch pages.
- indexer: parse and index pages.
- ranking engine: score and rank results.
- query processor: process user query.
- logs/feedback: collect interactions.

Past papers 多次把 crawling、freshness、politeness 和 ranking quality 放在一起考。

15.2 14.2 User Needs

Pair: Web Search → User intent [Past]

三类经典 web search needs:

- Informational: 找信息。
- Navigational: 去特定网站/page。
- Transactional: 完成行动, 如购买、下载、订票。

2023 Q1(h) 直接考过。答题最好给一个例子。

15.3 14.3 SEO and Spam

Pair: Web Search → SEO

SEO 是优化网页以提高 search ranking。合法 SEO 可以改善页面结构和内容; spammy SEO 则 manipulates ranking signals, 例如 keyword stuffing、cloaking、misleading meta-tags。

考试要会区分 algorithmic search results 和 sponsored ads。

15.4 14.4 Duplicate and Near-Duplicate Detection

Pair: Web Search → Duplicate detection

Strict duplicate detection 找完全相同 documents。Near-duplicate detection 找高度相似但不完全相同的 pages, 例如 mirror pages、模板化网页、轻微修改内容。

重要性: 减少 index pollution、节省 crawling/indexing resources、避免 search results 中重复页面。

15.5 14.5 Shingles and MinHash

Pair: Web Search → Shingles [Past]

Shingle 是 document 中连续 k -gram, 可以是 word shingles 或 character shingles。用 shingle sets 的 Jaccard similarity 判断 near-duplicate。

问题: 不能对 web-scale 所有 document pairs 精确比较。MinHash/sketch 用较小 signatures 近似 Jaccard similarity, 提高效率。

2023 Q1(e) 直接问过 shingle。

15.6 14.6 Web Crawling

Pair: Web Search -> Crawling workflow [Past]

Crawler 从 seed URLs 开始, fetch pages, parse links, normalize URLs, 把 unseen URLs 加入 URL frontier, 并遵守 robots.txt/politeness constraints。

还需要 revisit pages 以保持 freshness。

15.7 14.7 Polite Crawler

Pair: Web Search -> Politeness [Past]

Polite crawler 不会过度请求同一 host。它遵守 robots.txt、crawl-delay, 使用 per-host waiting time 或 back queues, 避免给服务器造成压力。

2023 Q1(f) 和 2024 Q1(d) 都涉及。

15.8 14.8 URL Frontier and Back Queues

Pair: Web Search -> URL frontier scheduling [Past]

URL frontier 管理待抓取 URLs。通常要同时考虑 priority 和 politeness。

Back queues 常按 host/site 组织 URLs, 确保同一 host 请求之间有间隔。Front queues 更偏 priority; back queues 更偏 scheduling/politeness。

2023 Q1(j) 直接考过 back queues。

15.9 14.9 Freshness and Hacked Pages

Pair: Web Search -> Freshness/security [Past]

Freshness 是 index 是否反映网页当前内容。Hacked pages 会污染 index, 可能让 search engine 返回 spam/malware/irrelevant content。

Mitigation 依赖 recrawling/change detection、content comparison、security signals、reputation signals。2025 Q1(f) 考过 hacked content 对 crawler/index 的影响。

16 15. Text Classification

16.1 15.1 Classification Types

Pair: Classification -> Binary/multi-class/multi-label [Past]

Binary classification: 两类, 如 spam/not spam。

Multi-class: 多类中选一个, 如 topic = politics/sport/tech。

Multi-label: 一个 item 可有多个 labels, 如一篇文章同时是 politics 和 economy。

2025 Q4(d) 直接问过 multi-class vs multi-label。

16.2 15.2 Rule-Based vs Supervised Classification

Pair: Classification -> Rule-based vs ML

Rule-based classifier 用人工规则, 优点是 interpretable、无需训练数据; 缺点是 brittle、coverage 差。Supervised ML 从 labelled data 学模式, scalable, 但需要数据, 可能学到 bias。

考试如果问 sentiment dictionary vs ML, 本质就是这一区域。

16.3 15.3 Feature Extraction

Pair: Classification -> Text features

文本分类要把 text 转成 vectors。常见 features:

- BOW counts
- TF-IDF weights
- word n-grams
- character n-grams
- topic features
- embeddings/contextual representations

Character n-grams 对 misspellings、social media、morphologically rich languages 有帮助。

16.4 15.4 Feature Selection

Pair: Classification -> MI/chi-squared feature selection [Past]

Feature selection 选择对 class 有区分力的 features, 减少维度和 noise。MI 和 chi-squared 都可衡量 term-class association。

Past papers 的 expected mutual information 计算题和 classification feature selection 是连在一起的。

16.5 15.5 Feature Scaling

Pair: Classification -> Feature scaling [Past]

Feature scaling 把不同 features 调到 comparable ranges, 防止数值大的 feature dominate model。对 SVM、logistic regression、distance-based methods 尤其重要。

2023 Q6(d) 直接问过 feature scaling。

16.6 15.6 Train / Development / Test

Pair: Classification -> Data splits [Past]

Training set 用来 learn model。Development/validation set 用来 tune hyperparameters 和 choose model。Test set 只用于 final unbiased evaluation。

常见坑: 在 test set 上调参是 cheating, 会让最终分数过于乐观。

2023 Q6(e) 和 2025 Q4(e) 都考过。

16.7 15.7 Evaluation under Imbalance

Pair: Classification -> Precision/recall for imbalanced data [Past]

如果 positive class rare, accuracy 可能 misleading。比如 misinformation detection 中, 大多数 posts 不是 misinformation, 一个总预测 “not misinformation” 的 classifier accuracy 可能很高但 useless。

如果 false negatives 代价高, 应强调 recall 或 F_β with $\beta > 1$ 。2023 Q8 就是这个逻辑。

16.8 15.8 Bias and Fairness

Pair: Classification -> Bias in sentiment classifier [Past]

模型可能把 demographic identity terms 和 negative/positive labels 建立 spurious association。例如训练数据中某些 identity terms 常出现在 toxic contexts, 模型会错误地认为 identity term 本身带有负面 sentiment。

缓解方法:

- Counterfactual data augmentation。
- Balanced training data。
- Bias evaluation slices。
- Debiasing or fairness constraints。
- Human-in-the-loop review。

答题要同时讲 advantages/disadvantages。

16.9 15.9 Error Analysis

Pair: Classification -> Error analysis

Error analysis 是系统性检查 misclassified examples, 找出 failure patterns, 例如 negation、sarcasm、OOV、domain shift、class ambiguity。

它不是只列几个错例, 而是用错例指导下一轮 feature/model/data improvement。

16.10 15.10 Zero-Shot and Transfer Learning

Pair: Classification -> Zero-shot / transfer learning [Past]

Transfer learning 用 pre-trained models 的 representations, 再在 task data 上 fine-tune。它缓解 OOV 和 small data 问题, 因为 model 已从大规模语料学到词义和上下文。

Zero-shot classification 不用 task-specific training examples, 而用 label descriptions/prompts 直接预测。优点是快速、低资源; 缺点是稳定性和 domain-specific accuracy 可能不足。

17 16. Modern IR, LLMs, and RAG

17.1 16.1 LLMs vs Search Engines

Pair: Modern IR → LLMs complement search [Past]

Past papers 2023/2025 都问过 LLM 是否会取代 search engines。强答案通常是 disagree with full replacement, 但承认 LLMs change the interface。

Search engines 仍重要, 因为它们提供 freshness、coverage、attribution、navigational access、transactional access。LLMs 擅长 synthesis、explanation、conversation。

考试模板:

LLMs complement rather than fully replace search. Retrieval provides fresh, attributable evidence; generation provides synthesis and interaction.

17.2 16.2 Freshness and Attribution

Pair: Modern IR → Freshness/attribution

LLM parameters 更新慢, 不能可靠存储最新 web state。Search index 可持续 crawl/re-index。Search results 还提供 URLs/snippets, 让用户 verify sources。

高风险领域如 medical/legal/news especially need attribution。

17.3 16.3 Dense Retrieval

Pair: Modern IR → Dense retrieval

Dense retrieval 用 encoders 把 queries 和 documents 映射到 embedding vectors, 然后用 vector similarity 检索。它能处理 semantic similarity, 即使 query 和 document 没有 exact term overlap。

缺点: 可解释性弱、需要向量索引、可能受 embedding model domain mismatch 影响。

17.4 16.4 RAG

Pair: Modern IR → Retrieval-Augmented Generation [Past]

RAG 先 retrieve evidence, 再 generate answer。它结合:

- Parametric memory: model weights 中的知识。
- Non-parametric memory: external index/vector database 中的 documents。

RAG 的核心价值是 grounding: 减少 hallucination, 提供 up-to-date/context-specific evidence。

17.5 16.5 Query and Document Encoders

Pair: Modern IR → Encoders in RAG [Past]

Dense RAG 通常有 query encoder 和 document encoder。Documents 的 embeddings 会预先计算并存入 vector database; query embedding 在 query time 计算。

如果更新 document encoder, 旧的 document embeddings 就 stale, 需要重新 embed 整个 corpus 并 rebuild index。这在 large-scale system 中非常贵。

2025 Q1(g) 直接考过为什么 not practical to update document encoder。

17.6 16.6 Vector Database Contents

Pair: Modern IR -> Vector DB documents [Past]

Coding assistant RAG 的 vector DB 不应只存 generic training data, 而应存 current project-specific documents:

- source code files
- README/docs
- config files
- tests
- API specs
- dependency metadata
- recent local project context

2025 Q6(b) 直接考过。

17.7 16.7 Privacy-Preserving RAG

Pair: Modern IR -> Local/private deployment [Past]

如果 users 担心代码通过 API 发到外部服务, 可以改成 local LLM 或 self-hosted model, 并把 embeddings/vector DB 保存在本地或 organization private infrastructure。

缺点: 成本更高、模型能力可能弱于 frontier API、维护复杂、hardware requirements 高。

18 17. Past-Paper Selection Logic

18.1 17.1 The Pattern Behind 2023 Q1

2023 Q1 的 short-answer list 实际上覆盖了多个 topics:

- Evaluation -> Cranfield test collection
- Language Model -> smoothing
- Web/PageRank -> PageRank meaning
- Feedback -> explicit relevance feedback
- Web duplicate detection -> shingle
- Crawling -> polite crawler
- Implicit feedback -> eye tracking
- Web search -> user needs
- Query logs/dense retrieval -> similar queries without shared words
- URL frontier -> back queues

这说明 short-answer 题倾向于跨 topics 抽 atomic concepts, 而不是只考某一讲。

18.2 17.2 Calculation Questions Pool

高频计算池:

- PageRank iterations
- TF-IDF / IDF / ranking
- Jaccard coefficient
- Jelinek-Mercer smoothing
- Precision / recall / F1
- R-precision
- AP / MAP
- MRR
- DCG / nDCG
- Expected mutual information
- Delta/v-byte decoding

复习原则: 每个公式都要会写 variables, 且能在 small toy example 上手算。

18.3 17.3 Short-Answer Template

遇到 “What is X?”, 用三句:

1. Definition: X 是什么。
2. Function: X 解决什么问题或为什么重要。
3. Example/caveat: 给一个例子或限制。

例如 shingle:

A shingle is a contiguous k -gram of tokens or characters from a document. Shingle sets are used to compare near-duplicate documents, often via Jaccard similarity or MinHash. The choice of k matters: too small gives many false matches, too large misses small edits.

18.4 17.4 Compare Question Template

遇到 “Compare X and Y”, 不要写两个孤立定义, 要按 dimensions 对比:

- assumption
- input/output
- advantage

- limitation
- typical use case

例如 pLSI vs LDA:

Both model documents as mixtures of latent topics, but pLSI has document-specific parameters tied to the training set, while LDA places a Dirichlet prior over document-topic distributions, giving a more complete generative model and better handling of unseen documents.

18.5 17.5 Calculation Answer Template

计算题写法:

1. 写公式。
2. 定义变量。
3. 代入数值。
4. 给三位小数。
5. 写一句 interpretation。

不要只给最终数字。TTDS past papers 明确说 calculations should show steps。

19 Final Revision Checklist

考前你应该能快速回答:

- 什么是 Cranfield paradigm?
- 为什么 smoothing 重要?
- PageRank 值代表什么?
- Polite crawler 和 back queues 分别是什么?
- Shingle 用来干什么?
- TF、DF、CF、IDF 有何区别?
- TF-IDF / cosine / LM 各自如何 ranking?
- P@k、R@k、F1、R-precision、AP、MRR、nDCG 怎么算?
- MI contingency table 怎么画?
- pLSI 和 LDA 最大区别是什么?
- Gibbs sampling 怎么重新分配 topic?
- Web search 三类 user needs 是什么?
- Explicit vs implicit feedback 有何区别?
- Train/dev/test 为什么要分开?
- Multi-class vs multi-label 有何区别?
- LLM 为什么不会简单取代 search?
- RAG 里的 vector DB 应该存什么?